

National University of Computer and Emerging Sciences, Lahore Campus

Course:	Design and Analysis of Algorithms
Program:	BS(Data Science)
Duration:	30 Minutes
Paper Date:	31-Oct-2023
Section:	5A
Exam:	Quiz 3

Course Code:	CS2009
Semester:	Fall 2023
Total Marks:	11
Weight	%
Page(s):	4

Q1) The longest increasing subsequence problem is to find a subsequence of a given sequence in which the subsequence's elements are in sorted order, lowest to highest, and in which the subsequence is as long as possible. This subsequence is not necessarily contiguous or unique. [6 Marks]

Please note that the problem specifically targets subsequences that need not be contiguous, i.e., subsequences are not required to occupy consecutive positions within the original sequences.

For example, the longest increasing subsequence of [0, 8, 4, 12, 2, 10, 6, 14, 1, 9, 5, 13, 3, 11, 7, 15] is [0, 2, 6, 9, 11, 15].

Following is a brute force recursive solution with exponential time complexity. For each item, there are two possibilities:

1. Include the current item in LIS if it is greater than the previous element in LIS and recur for the remaining items.
 2. Exclude the current item from LIS and recur for the remaining items.
- ```
1. int LIS(int arr[], int i, int n, int prev)
2. {
3. // Base case: nothing is remaining
4. if (i == n) {
5. return 0;
6. }
7. // case 1: exclude the current element and process the
8. // remaining elements
9. int excl = LIS(arr, i + 1, n, prev);
10. // case 2: include the current element if it is greater
11. // than the previous element in LIS
12. int incl = 0;
13. if (arr[i] > prev) {
14. incl = 1 + LIS(arr, i + 1, n, arr[i]);
15. }
16. // return the maximum of the above two choices
17. return max(incl, excl);
18. }
19. int main()
20. {
21. int arr[] = { 0, 8, 4, 12, 2, 10, 6, 14, 1, 9, 5, 13, 3, 11, 7, 15 };
22. int n = sizeof(arr) / sizeof(arr[0]);
23. cout << "The length of the LIS is " << LIS(arr, 0, n, INT_MIN);
24. return 0;
25. }
```

Name: \_\_\_\_\_ Reg #: \_\_\_\_\_ Section: \_\_\_\_\_

Write an iterative dynamic programming solution (pseudocode) for this problem. Analyze its time complexity.

**Solution**

**<https://www.geeksforgeeks.org/longest-increasing-subsequence-dp-3/>**

Name: \_\_\_\_\_

Reg #: \_\_\_\_\_

Section: \_\_\_\_\_

**Q2)** Dry run longest common subsequence (LCS) dynamic programming algorithm on the following sequences:

X = TACGC

Y = GCATAGC

Fill the following table. Each cell contains length of LCS of a subproblem. Place arrows pointing to the previous cell which is used for defining values of current cell. [1 + 4 = 5 Marks], 1 mark for writing correct values, 4 marks for placing correct arrows.

| X<br>Y | T | A | C | G | C |
|--------|---|---|---|---|---|
| G      |   |   |   |   |   |
| C      |   |   |   |   |   |
| A      |   |   |   |   |   |
| T      |   |   |   |   |   |
| A      |   |   |   |   |   |
| G      |   |   |   |   |   |
| C      |   |   |   |   |   |

**LCS Algorithm**

LCS-Length(X, Y)

1.  $m = \text{length}(X)$  // get the # of symbols in X
2.  $n = \text{length}(Y)$  // get the # of symbols in Y
3. for  $i = 1$  to  $m$        $c[i,0] = 0$       // special case:  $Y_0$
4. for  $j = 1$  to  $n$        $c[0,j] = 0$       // special case:  $X_0$
5. for  $i = 1$  to  $m$       // for all  $X_i$
6.      for  $j = 1$  to  $n$       // for all  $Y_j$
7.           if (  $X_i == Y_j$  )
8.                 $c[i,j] = c[i-1,j-1] + 1$
9.           else
10.                 $c[i,j] = \max( c[i-1,j], c[i,j-1] )$
11. return c